ACCESCO LIVING PRIVATE LIMITED

Customer-Facing Replenishment and Reminder Engine

Step-by-step guide to estimate when a customer may run out of a product and send appropriate reminders

Plain-language, step-by-step implementation manual
No code snippets; clear gates, owners and acceptance checks

Document ID	ACC-REPL-CUST-IMPL-02
Version	Revision 2.0 - plain-language edition
Status	Implementation proposal - requires system-owner approval
Prepared for	Data & Analytics, Engineering, Operations and system owners

Important: this is a build guide, not evidence that the proposed APIs, data fields, systems or model performance already exist.

Contents

- 1. Executive summary and scope
- 2. What is confirmed and what must wait
- 3. Customer-engine architecture, explained layer by layer
- 4. How this engine differs from warehouse replenishment
- 5. Responsibilities of OMS, IMS, WMS and TMS
- 6. Data requirements by source and processing layer
- 7. Technology and people needed
- 8. Step-by-step build plan from an empty workspace
- 9. How the time-to-depletion model works
- 10. Customer example and cold-start handling
- 11. Testing plan and acceptance checks
- 12. Reminder rules, privacy and customer controls
- 13. Deployment, rollback and ongoing monitoring
- 14. Main risks and decisions required
- 15. Final go-live checklist
- References and supporting documents

1. Executive summary and scope

The customer-facing engine estimates when a particular customer may be close to running out of a product they previously bought. For example, it may estimate that a customer who regularly buys a 1-litre cooking oil pack may need another one soon. The system can then prepare a reminder candidate, but only if the customer has agreed to receive it and the product is currently available.

This is different from predicting what a store should order. The unit of prediction is a customer and a product, using that person's qualifying purchase history. It does not forecast warehouse demand, reserve stock, create an order or guarantee the date a product will be empty.

Starting condition: Accesco currently has SKU catalogue and inventory data in PostgreSQL, while customer accounts and linked order history are not confirmed as available. The data collection and consent design can begin now; a personalised model and customer reminders must wait until a sufficient, reliable and approved history exists.

Core outputs

- A predicted time window in which a previously purchased product may need replenishment, along with uncertainty.
- A reason the customer-product pair was considered eligible, without exposing sensitive personal details.
- A reminder candidate that is checked again for customer consent, stock availability, recent purchases, current cart and message limits.
- An outcome record that shows whether the reminder was sent, suppressed, delivered, clicked, followed by a purchase, or followed by an opt-out.

2. What is confirmed and what must wait

Area	Current status	Required decision / evidence
SKU catalogue	Exists in PostgreSQL.	Confirm product ID, variant, pack size, unit, category and product-substitution mapping.
Customer account identity	Not confirmed as available for analytics.	Confirm approved, stable customer identifier and permitted use of purchase history.
Historical customer orders	Limited or unavailable.	Confirm fulfilled order lines with customer, SKU, actual quantity and fulfilment/delivery time.
Actual product depletion	Likely not directly observed.	Decide whether a measurable depletion signal exists. If not, clearly label next qualifying purchase as a proxy.
Consent and message preferences	Not confirmed.	Define the purpose, consent wording, opt-out behavior, permitted channel, quiet hours and frequency cap.
Availability data	Inventory exists, but API and freshness are TBC.	Must be checked immediately before sending; a reminder for unavailable stock is a poor experience.
Model family	XGBoost Survival AFT is the selected candidate.	Train only after there are enough linked examples and a valid definition of the event. Otherwise use manual reminders or no personalized prediction.
Customer cold start	New customers may have no purchase history.	Do not produce a personalised learned estimate when evidence is insufficient; allow an explicit user-set reminder if product design approves it.
Initial execution	Human-approved operational approach; customer message release requires product/privacy approval.	Confirm whether reminders will be automatically sent after the launch policy is approved, and who can pause them.
Team, dates and budget	Yet to be assigned / TBC.	Assign OMS, IMS, product, backend, privacy and analytics owners.

3. Customer-engine architecture, explained layer by layer

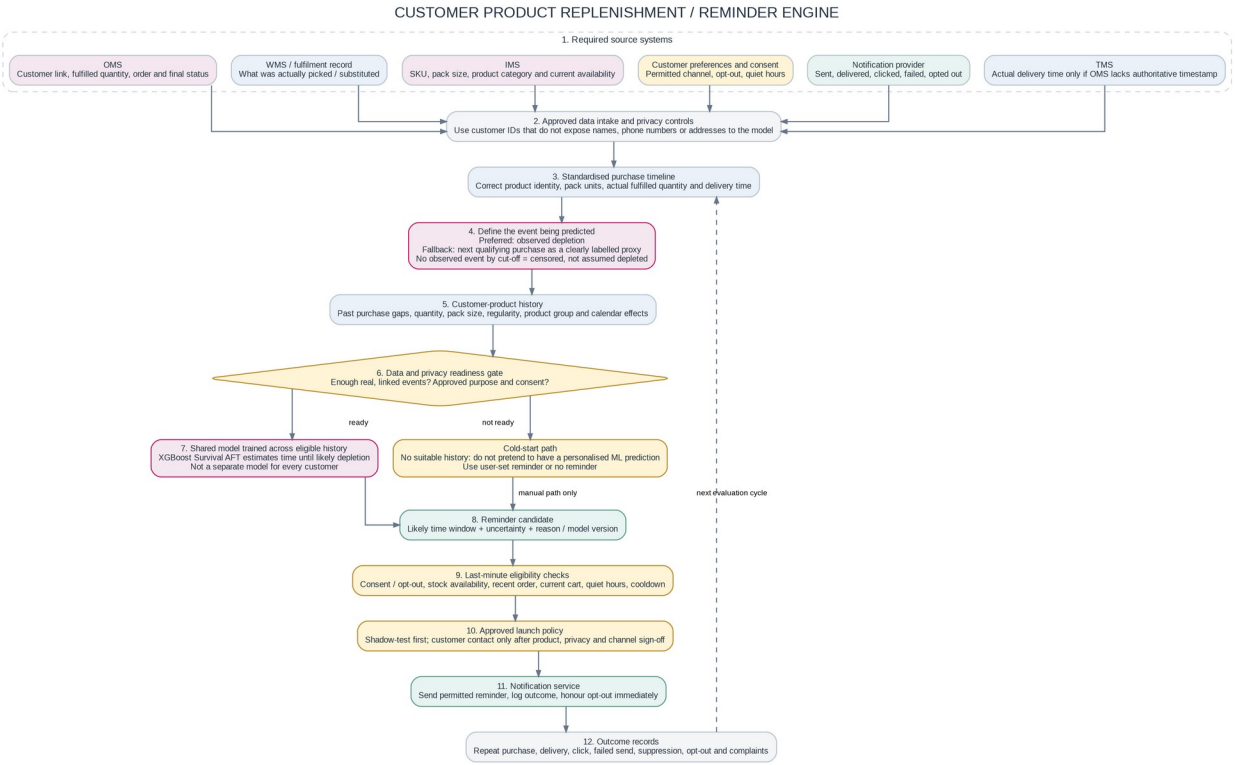


Figure 1. Customer engine: purchase history is standardised, the event is defined honestly, prediction is followed by consent and availability gates.

The main safety feature of this design is the eligibility check after prediction. The model is allowed to estimate timing; it is not allowed to bypass consent, message preferences, product availability or duplicate-reminder controls. Customer identity is minimised in the analytical data so that a model does not need names, phone numbers or addresses.

Layer	Plain-language job	What it produces	Key protection
1. Source facts	Read completed orders, actual picked quantities, product units, availability and consent settings from their owners.	An event record for each qualifying customer-product purchase and the latest permission status.	Only approved fields are collected.
2. Intake and identity protection	Move the needed records into analytics and replace direct identity fields with controlled customer keys.	Restricted original records, load history and a pseudonymous identifier.	Keep direct contact details out of model inputs.
3. Standardised purchase timeline	Resolve pack sizes, quantities, product IDs, returns and actual fulfilment/delivery time.	A consistent timeline for each customer and product.	Do not use cancelled orders as consumption.
4. Event definition	Define what event is being predicted and whether it has actually been observed.	Either observed time-to-depletion or a clearly labelled next-purchase proxy; unresolved cases marked as unobserved by the cutoff.	Do not label every customer with no repeat purchase as depleted.
5. Customer-product features	Summarise past purchase gaps, pack quantity, regularity and relevant calendar patterns.	A learning record for each eligible customer-product observation.	No unnecessary sensitive or unrelated personal information.
6. Model and evaluation	Train a shared model across eligible history and test whether timing estimates are useful.	Time window and uncertainty estimate plus model version.	The result is not proof of actual depletion.
7. Eligibility and message rules	Check whether a reminder is appropriate right now.	Eligible, suppressed or expired reminder candidate with a reason.	Consent, availability, recent order, cart, quiet hours and cooldown must be enforced.

Layer	Plain-language job	What it produces	Key protection
8. Notification and feedback	Send through an approved channel and record the outcome.	Sent/failed/delivered/clicked/opt-out and later purchase events.	Opt-out takes priority; retries must not send duplicates.
9. Monitoring and review	Review false reminders, missed opportunities, customer impact and model drift.	Performance and safety report.	Pause the customer-facing service if consent or availability checks fail.

4. How this engine differs from warehouse replenishment

Question	Warehouse / store engine	Customer-facing engine
Who is the prediction about?	One SKU at one warehouse/store and time period.	One eligible customer and one product for a purchase episode.
What is predicted?	Units likely to be sold by a future time; then a proposed stock action.	Time until likely depletion or the next qualifying purchase proxy.
Main data	Sales, available stock, reservations, incoming supply, lead time, pack size, expiry and capacity.	Customer-linked fulfilled purchases, actual quantities, product pack size, purchase gaps, consent and reminder outcomes.
Main model candidate	Forecasting method compared with baselines.	XGBoost Survival AFT after enough customer-product events exist.
Main business rules	Inventory position, safety buffer, supplier minimum, pack multiple, storage and expiry.	Consent, opt-out, current availability, recent purchase, current cart, quiet hours and message frequency.
Main action	Propose stock quantity and timing for staff approval.	Prepare or send an allowed product reminder under an approved contact policy.
What it must never do	Invent inventory or directly change stock without approval.	Claim certain depletion, bypass consent, or automatically place an order for the customer.

5. Responsibilities of OMS, IMS, WMS and TMS

System	Needed information	Why it matters to customer estimates	Boundary
OMS - Order Management System	Customer link or approved pseudonymous key, order line, ordered quantity, final fulfilment state, cancellation/return/substitution, and order timestamps.	Main source for linked purchase history and the customer's past product purchases.	Count completed/fulfilled purchases, not every order attempt.
WMS - Warehouse Management System	Actual item picked, substituted or short-picked; final packed/fulfilled quantity; fulfilment timestamp when OMS does not retain it.	The customer may receive a different quantity or product from what was originally ordered.	Use actual fulfilled quantity; preserve substitution links.
Inventory IMS	SKU/product master, pack size and unit conversions, category, current stock/availability and active/inactive status.	Makes customer history comparable across product packs and prevents reminders for unavailable products.	IMS availability must be rechecked at send time. It does not own customer order history in the proposed map.
TMS - Transport Management System	Actual delivery event/time only if OMS does not store an authoritative delivered timestamp; delivery failures if they affect final fulfilment.	Consumption is more plausibly estimated from when a product was delivered, not when checkout started.	Confirm whether the TMS event is reliable and linked to the order.
Consent / notification service	Consent purpose, channel preference, opt-out, delivery result, complaint and message history.	Prevents unwanted, duplicate or inappropriate reminders.	The model must not be able to override contact restrictions.

6. Data requirements by source and processing layer

Data group	Required information	Why it is needed	If missing
Customer key	Stable customer ID from OMS, converted to an approved pseudonymous key for analytics.	Links purchases by the same customer without exposing direct identity.	No personalised model for unlinked records.
Qualifying purchase	Order and line ID, customer key, SKU, quantity ordered, quantity actually fulfilled, status, event time and location/vertical if useful.	Defines the beginning of each product-purchase episode.	Do not use the row until status and quantity can be established.
Delivery / handover time	Authoritative final delivery/fulfilment timestamp where relevant.	Starts the likely consumption window closer to when the customer received the product.	Use only an approved fallback timestamp and record the limitation.
Product / unit mapping	SKU, variant, pack size, unit, quantity conversion, multipack size, product category and substitution relationship.	Two packs of different sizes do not represent the same quantity of product.	Do not combine incompatible packs or estimate quantity from product names alone.
Returns and cancellations	Cancelled lines, refunds, returned items, failed delivery and substitutions.	These events can make an apparent purchase invalid or alter the quantity the customer received.	Exclude or adjust according to agreed rules; do not count them as normal consumption.
Repeat-purchase history	Previous purchase events and elapsed time between qualifying purchases for the same customer and product group.	Helps estimate personal purchasing rhythm.	Cold-start path; no personalised timing claim.
Product availability	Current sellable status and location/vertical availability; freshness timestamp.	A reminder should not lead the customer to an unavailable product.	Suppress reminder if availability cannot be checked reliably.
Consent and preferences	Permission, purpose, channel, opt-out, quiet hours, language preference if provided, send history and cooldown.	Defines whether and how customer contact is allowed.	Suppress contact; do not infer permission from account creation or past purchases.
Reminder outcomes	Candidate ID, eligibility reason, sent time, channel, delivery/failure, click, repeat purchase, opt-out and complaint if available.	Allows the team to improve timing and monitor customer harm or annoyance.	Do not activate broad sending without an auditable outcome trail.
Context (optional)	Day of week, season, elapsed time, product category and pack quantity.	Can improve timing estimates without requiring unnecessary personal details.	Start with fewer well-defined fields rather than inventing customer lifestyle features.

Privacy rule: Do not use raw customer names, email addresses, phone numbers or full delivery addresses as model features. Keep direct contact details in the approved customer/notification service, and use a limited customer key in analytical training data.

7. Technology and people needed

Need	Recommended starting choice	Why
Source records	Existing OMS/IMS/WMS/TMS interfaces and PostgreSQL.	Use the existing system owner for each fact; do not create a parallel customer history ledger.
Analytics and storage	Databricks controlled tables with restricted access for customer-linked events.	Provides a repeatable place to prepare timelines and evaluate models.
Data preparation	Databricks jobs and managed transformations.	Keeps customer events, units, returns and labels consistent each run.
Model candidate	XGBoost Survival AFT, once enough event data exist.	Estimates time to an event and can handle cases where the observation ends before the event is seen.
Benchmark	Simple repeat-purchase interval or category-level estimates used as a comparison, not represented as a trained personalised model.	Shows whether the advanced model actually improves timing.

Need	Recommended starting choice	Why
Model tracking	MLflow / governed model registry if available.	Records candidate version, data window, evaluation and release decision.
Serving / scheduling	Scheduled candidate generation initially; a small authenticated API or approved backend integration when needed.	Avoids building a complex live service before customer history volume requires it.
Message delivery	Accesco-approved notification service, with push / WhatsApp / other channels only after product and privacy approval.	Centralises consent, retries, cooldown and channel preferences.
Security	Restricted customer-data access, pseudonymous IDs, consent checks, audit trail and limited retention.	Protects customer history and enforces permitted use.
People	OMS owner, IMS owner, product owner, backend/notification engineer, data engineer, ML lead, privacy reviewer.	Each must approve the portion they own; all names are currently TBC.

8. Step-by-step build plan from an empty workspace

Step 1. Agree the customer benefit and contact policy

Purpose: Make sure the reminder solves a real customer problem and is permitted.

What to do:

- Define the product groups to support first and the intended reminder benefit.
- Decide which channels may be used and whether the customer must explicitly opt in.
- Agree how a customer opts out, changes preferences, pauses messages and complains.
- Define quiet hours, frequency caps, cooldown after a reminder, and when a recent purchase suppresses another message.
- Name the product, privacy and operations approvers. No reminder should be sent before they approve the policy.

Expected output: An approved use-purpose, consent and customer-message policy.

Do not move ahead until: Without approved consent and contact rules, data collection and offline model design may continue but customer messaging stays disabled.

Step 2. Check that customer history can be linked safely

Purpose: Confirm the data supports customer-product prediction.

What to do:

- Ask OMS for completed order lines, customer IDs, final quantities, timestamps, cancellation/return and substitution status.
- Ask WMS which record identifies the actual picked and fulfilled items.
- Ask IMS for pack size and unit conversions so purchases can be compared fairly.
- Confirm where the actual delivery timestamp lives. Prefer the authoritative timestamp rather than checkout time.
- Ask the privacy owner whether these records can be used for the stated reminder purpose and how long they may be retained.

Expected output: A field-by-field source map plus privacy approval or an explicit list of blockers.

Do not move ahead until: If customer identity cannot be linked or permission is not approved, do not create a personalised training set.

Step 3. Create restricted intake and identity protection

Purpose: Collect only what the model needs.

What to do:

- Use approved read-only source access and separate the contact details from analytical records.
- Replace direct identity with a controlled customer key before preparing features.
- Keep a record of data load times and rejected rows without copying direct contact details into general logs.
- Limit who can view raw customer history and retain only the minimum duration approved by policy.
- Test that an opt-out or data-correction request can be propagated to derived tables and candidate queues.

Expected output: Restricted source extracts and a tested data-protection process.

Do not move ahead until: Do not move data into model training if deletion, correction or consent requirements have no owner.

Step 4. Standardise the purchase timeline

Purpose: Make each product purchase event mean the same thing.

What to do:

- Keep only qualifying purchases according to the approved completion rules.
- Use final fulfilled quantity, including partial fulfilment or substitutions where the source can prove them.
- Convert pack sizes into comparable units; never treat a 500 ml pack as equivalent to a 1 litre pack.
- Choose the event timestamp and keep its definition consistent across every customer and product.
- Apply a documented rule to cancelled lines, refunds, returns and failed deliveries.
- Reconcile a sample of customer histories with OMS and WMS records.

Expected output: A clean chronological timeline with row-level reasons for exclusions and corrections.

Do not move ahead until: Do not create model labels from raw order placement dates or unverified quantities.

Step 5. Choose what “running out” means

Purpose: Define the event before training any model.

What to do:

- First check whether a real depletion signal exists, such as a customer-reported depletion or another directly observed event.
- If direct depletion is not recorded, decide whether the next qualifying purchase can be used as a proxy; make the limitation visible in product documentation.
- Define what makes a repeat purchase qualifying: same product, acceptable substitution, quantity changes, and treatment of returns.
- For customers whose observation period ends before a qualifying event, record that the event was not observed by the cutoff. Do not invent a depletion date.
- Get the product and analytics owners to approve the label definition.

Expected output: A written label definition with examples for observed, unobserved and ambiguous cases.

Do not move ahead until: No AFT model training until the target/event and unobserved-case handling are approved.

Step 6. Build the customer-product learning records

Purpose: Prepare model inputs from past purchase patterns.

What to do:

- Build a separate observation for each eligible customer-product purchase episode.
- Include only information available at the time a prediction would have been made.
- Summarise previous purchase gaps, fulfilled quantity, pack size, product category and relevant calendar patterns.
- Keep current consent and contact preferences in the eligibility service; they are not a reason to expose direct contact details to the model.
- Check how many observed repeat-purchase events exist and how many cases are censored or ambiguous.

Expected output: A reviewed data-readiness report for eligible examples, event counts and exclusions.

Do not move ahead until: Do not assume there is enough training data simply because the number of order lines is large; repeat events and valid customer links matter.

Step 7. Build a simple baseline

Purpose: Create an honest point of comparison.

What to do:

- For eligible products/customers with repeat history, calculate a simple historical purchase interval or robust group-level interval.
- Show how much those simple estimates vary by product, pack size and customer history.
- Check how often a baseline would suggest a reminder before an actual repeat purchase, and how often it would be too early or too late.
- Keep new customers without history on a separate path such as user-set reminders or no personalised estimate.

Expected output: A baseline report showing strengths, limitations and eligible population.

Do not move ahead until: Do not market the baseline as a learned personal prediction.

Step 8. Train and compare XGBoost Survival AFT

Purpose: Use the selected model only if the data are suitable.

What to do:

- Train one shared model across a sufficiently large set of eligible customer-product examples; do not attempt a separate model for each customer.
- Use the approved observed/proxy event time and the marker that says whether the event was seen or not.
- Test different model settings using only data available before each evaluation cutoff.
- Keep customer histories or later time periods out of training when they would leak future information into evaluation.
- Compare the model with the baseline and check whether it gives useful timing estimates across product groups and history lengths.
- Record uncertainty and limits, including that purchases from other retailers are generally invisible.

Expected output: A reproducible model evaluation and a decision to promote, revise or defer the candidate.

Do not move ahead until: If there are too few valid events, keep collection running and do not claim model readiness.

Step 9. Turn a prediction into a reminder candidate

Purpose: Keep prediction separate from contacting the customer.

What to do:

- For each eligible customer-product pair, save a predicted time window, model version, calculation time and the reason it was considered.
- Use an approved time window rather than presenting a false exact depletion date.
- Give each candidate an expiry time so stale candidates cannot be sent days later without rechecking.
- Apply a minimum-history policy; proposed starting point is to wait for several qualifying purchases (for example, 3-5), then validate this threshold on real data.
- Allow a candidate to be suppressed if the data or prediction is uncertain.

Expected output: A candidate queue containing only prediction outputs, not permission to send.

Do not move ahead until: A predicted date must never bypass the eligibility rules or become an order.

Step 10. Apply all reminder eligibility checks

Purpose: Protect customer experience at the moment of sending.

What to do:

- Check current consent and opt-out immediately before sending.
- Check that the product is active and available through IMS or the approved inventory API.
- Suppress the reminder if the customer has recently bought the product, has it in the current cart, or an order for it is still in progress.
- Enforce quiet hours, channel preferences, cooldown and maximum frequency.
- Use a unique candidate/send reference and an atomic send-claim so retries cannot send duplicates.
- If a required eligibility service is down, suppress the message rather than assume permission or stock availability.

Expected output: A tested eligibility decision with a human-readable suppression reason for every blocked candidate.

Do not move ahead until: Any consent failure, duplicate send or send for a definitely unavailable product is a release blocker.

Step 11. Test in shadow mode

Purpose: See what would have been sent without contacting customers.

What to do:

- Generate candidates for a controlled time period, but keep actual sending disabled.
- Review candidate timing with product and operations staff against purchase history and product availability.
- Estimate false or too-early reminders and missed replenishment windows using the approved event definition.
- Check distribution by product, history length and customer eligibility; identify groups where the model has too little evidence.
- Review the notification templates and suppression reasons with the product owner.

Expected output: A shadow report and approval to proceed to a limited customer pilot.

Do not move ahead until: Do not use clicks or a single repeat purchase as proof that the depletion estimate is correct.

Step 12. Run a limited customer pilot

Purpose: Activate only after policy and data checks pass.

What to do:

- Use a limited, consented audience, a small selection of products and approved channels.
- Keep a manual pause switch that is independent of the model.
- Compare reminder outcomes with a holdout or baseline approach where product/privacy owners approve the experiment.
- Monitor opt-outs, complaints, failed sends, unavailability, recent-purchase suppressions and repeat purchases.
- Expand only after the product owner accepts both customer impact and operational results.

Expected output: A pilot review that records benefits, harms, limitations and a go/no-go decision.

Do not move ahead until: Pause contact if consent checks, stock availability or duplicate-suppression controls fail.

Step 13. Operate, monitor and refresh carefully

Purpose: Keep customer predictions useful without sending too many messages.

What to do:

- Monitor the quality and freshness of purchase records and product-unit mappings.
- Monitor time-window calibration, reminder timing, repeat purchases, opt-outs and complaints.
- Check whether product changes, packaging changes, promotions or stockouts distort the purchase timeline.
- Retrain only when enough new valid events exist and the candidate beats the current model on a fresh evaluation.
- Keep the current approved model, a no-send fallback and the manual reminder path available.

Expected output: A periodic model and customer-safety review.

Do not move ahead until: Never auto-promote a new model or widen messaging to a new group without approval.

9. How the time-to-depletion model works

XGBoost Survival AFT (Accelerated Failure Time) is designed for a question where the event takes place at an unknown future time. The important difference from ordinary forecasting is that some customers may not have had another qualifying purchase by the time the data are collected. For those records, the team knows the event had not been seen by the last observation date, but does not know when it will occur.

Record situation	How it should be recorded	What the model can learn
A valid depletion event is observed	Record the elapsed time from the agreed starting point to observed depletion.	A known time-to-event example.
Direct depletion is not measured, but a qualifying repeat purchase occurs	If approved, record the time to the next purchase as a proxy and label it as a proxy.	A known repeat-purchase interval, not proof of physical depletion.
No qualifying next purchase by the data cutoff	Record the length of observation and that the event was not observed by the cutoff.	An unobserved/censored example; do not give it a made-up event date.
Order was cancelled, returned or not delivered	Exclude or adjust according to approved fulfillment rules.	Do not treat it as normal consumption.
Pack size changed or product was substituted	Normalize units if a valid conversion exists; otherwise use a reviewed mapping or exclude.	Comparable examples only where product quantity is meaningful.

Record situation	How it should be recorded	What the model can learn
Only one purchase exists	No personal purchase interval is observable.	Keep it on a cold-start path until the agreed minimum history is met.

What counts as a good model?

- Its time windows align better with later observed/proxy events than the simple baseline.
- It handles records whose event is not observed without falsely labelling them as depletion.
- Its estimates are checked by product group and purchase-history length, not only as one overall average.
- It gives useful timing without increasing unwanted contact, opt-outs or complaints beyond approved limits.
- Its release notes state that the prediction is an estimate and document the limitations of the chosen event proxy.

10. Customer example and cold-start handling

Illustrative example only; not Accesco data. A customer repeatedly buys the same 1-litre cooking oil pack. The qualifying delivery timestamps are reasonably consistent, and actual fulfilled quantity is known. The model might estimate a likely replenishment window, for example “likely to need another pack within the next several days,” rather than asserting that the bottle will be empty at an exact hour.

Situation	Correct system behavior
Customer has several qualifying purchases, permission is active, and estimate is eligible.	Create a reminder candidate, then recheck availability, recent orders/cart, quiet hours and cooldown before sending.
New customer or only one qualifying purchase.	Do not claim a personalised learned estimate. Offer a user-set reminder only if the product supports it and the customer chooses it.
Customer has several purchases but no permission for reminders.	Suppress contact. Purchase history is not consent.
The product is currently unavailable.	Suppress the reminder or use a product-approved alternative path; do not promote an item that cannot be obtained.
Customer recently bought another pack or has one in a current order/cart.	Suppress the candidate to avoid a redundant reminder.
No repeat purchase is seen before data collection ends.	Treat as an unobserved event by the cutoff. Do not declare that the product was depleted or that the customer abandoned it.
Customer opted out after a reminder.	Immediately stop future sends according to policy and propagate the opt-out to any queued candidates.

Cold-start policy: Use no personalised model claim until enough reliable history exists. A proposed operational starting point is several qualifying purchases (for example, 3-5) before using personal purchase rhythm, but this threshold must be tested and approved. The model itself still needs a broad, suitable training dataset across many customers and products.

11. Testing plan and acceptance checks

Test stage	What to test	Evidence required to pass
Source/identity tests	Customer key stability, linked OMS lines, actual fulfilled quantities and delivery timestamps.	A sample of histories reconciles to source systems; no direct identity leaked into model features.
Unit and product tests	Pack size, multipack, variant, substitution, return, refund and quantity conversion.	Every eligible event has a defensible product/quantity interpretation; ambiguous cases are excluded or reviewed.
Event-label tests	Observed event, purchase proxy, no event by cutoff, cancelled order and return.	Each case receives the correct observed/unobserved status with a documented rule.

Test stage	What to test	Evidence required to pass
Model evaluation	Chronological evaluation, new versus repeat customers, product group, history length and comparison with baseline.	Results are reproducible and useful; no random split that leaks future purchase events.
Eligibility tests	No consent, opted out, quiet hours, recent purchase, cart/open order, unavailable product, cooldown.	Every blocked case is suppressed with a reason; no model result bypasses the gate.
Notification integration	Timeout, duplicate retry, expired candidate, invalid channel, provider failure and late opt-out.	No duplicate send; messages are suppressed or retried safely based on approved rules.
Privacy tests	Role access, logs, data export, correction/deletion path and retention.	Only approved roles can read data; opt-out/correction behavior meets policy.
Shadow run	Candidate timing and predicted reminder usefulness, with sending disabled.	Product/privacy owners approve the limited pilot based on evidence.
Rollback test	Kill switch, no-send mode, current model rollback and candidate queue stop.	Contact can be stopped quickly without affecting OMS orders or inventory.

Release blockers

- No approved purpose or valid consent handling.
- No reliable customer-to-fulfilled-product linkage or defensible event label.
- Censored observations are incorrectly treated as exact depletion dates.
- Reminder sends when the product is unavailable or the customer has opted out.
- Duplicate send during retries or inability to stop queued messages.
- No no-send fallback, audit trail, monitoring owner or rollback test.

12. Reminder rules, privacy and customer controls

Control	Required behavior	Why it exists
Consent and purpose	Store permission for the specific reminder purpose and permitted channel; check it again at send time.	Account registration or past purchase is not automatic permission.
Opt-out	Apply opt-out promptly and prevent any queued reminder from being sent afterward.	Customer choice overrides the prediction.
Availability	Check that the product is active and available immediately before send.	Avoid sending customers to an unavailable item.
Recent purchase / active order	Check recent fulfilment, an open order and current cart where the product exposes that information.	Avoid redundant reminders.
Quiet hours	Respect approved time windows and local timezone.	Avoid disruptive contact.
Cooldown and frequency	Set a maximum reminder frequency and wait period after a reminder.	Prevents repeated messages when the model is uncertain.
Candidate expiry	Drop or re-evaluate a candidate after its expiry time or when key facts change.	Avoid using a prediction based on old data.
Reason and audit	Record why candidate was created or suppressed, model version, eligibility check time and send outcome.	Allows support and compliance teams to explain behavior.
Data minimisation	Use pseudonymous customer keys in model data and keep contact details in the approved notification/customer service.	Reduces exposure of personal information.
Sensitive product handling	Review product categories that could reveal sensitive personal circumstances before personalization.	Avoid inappropriate inferences from purchase history.

13. Deployment, rollback and ongoing monitoring

Stage	What happens	Promotion gate
Data collection only	Load and validate historical fulfilled purchases; no model-based messages.	Privacy approval and quality of customer-product linkage are confirmed.
Offline test	Build labels, compare baseline and AFT candidate, assess timing and censoring.	Model shows useful performance on a fair holdout and limitations are documented.
Shadow mode	Create reminder candidates while message sending remains disabled.	Eligibility suppression, audit, templates and candidate expiry work correctly.
Limited pilot	Small consented audience and approved product group; monitor outcomes closely.	No critical consent/availability/duplicate failures; customer and product outcomes are acceptable.
Wider launch	Extend only to approved channels and eligible groups.	Product/privacy approvers sign off; kill switch and support are active.
Routine review	Monitor fresh data, message outcomes, opt-outs and timing quality; retrain only with enough new events.	Each model promotion is separately evaluated and approved.

Rollback and stop-send procedure

- Activate the no-send switch if consent, stock availability or notification eligibility checks are unavailable or incorrect.
- Stop the candidate-sending job and expire queued candidates that may be stale.
- Preserve the audit record for sent, suppressed and pending candidates.
- Return to the simple baseline or no-personalised-reminder path while the issue is investigated.
- Fix the source problem, repeat integration tests and get product/privacy approval before resuming sends.

Monitoring measures

Measure	What it tells you	How to interpret it carefully
Eligible history coverage	How many customers/products have enough valid history to estimate timing.	A low share is a data readiness issue, not a reason to invent predictions.
Timing quality	How well predicted windows align with observed event or approved repeat-purchase proxy.	Do not call proxy timing true physical depletion.
Suppression reasons	How many candidates are blocked by consent, availability, recent purchase or message limits.	High suppression may be correct and protective.
Repeat purchase after reminder	Whether a qualifying purchase occurs after the message.	It does not prove depletion and should be compared against a baseline/holdout.
Opt-outs and complaints	Signals that customers may find reminders unwanted or inaccurate.	Set explicit limits; pause if they rise beyond approved limits.
Send reliability	Failed sends, duplicate attempts, stale candidates and provider errors.	Any duplicate or post-opt-out send is a critical incident.
Data drift	Product or purchase patterns shift compared with model training data.	Re-evaluate model; do not auto-promote a replacement.

14. Main risks and decisions required

Risk / open question	Why it matters	Response / owner to confirm
No linked customer history	The model cannot learn customer-product intervals.	Implement data capture first; no personalised model claims. OMS/product owner TBC.

Risk / open question	Why it matters	Response / owner to confirm
No direct depletion signal	A repeat purchase is only a proxy for consumption.	Approve proxy label and explain the limits in product language.
Pack size or substitutions are unclear	The time until a 500 ml pack is depleted differs from a 1 litre pack.	Normalize only where the conversion is reliable; review or exclude ambiguous pairs.
Purchases from other retailers are invisible	Customer may not need another product despite no purchase from Accesco.	Keep prediction uncertain, use suppression and avoid over-contact.
Consent not built into data/events	Past order does not mean marketing permission.	Define the approved purpose, consent and opt-out flow before launch.
Availability not fresh	Customer receives a reminder for an unavailable product.	Recheck current availability before each send; suppress on service failure.
Small history per customer	Individual purchase rhythm may be unreliable.	Use a shared model and explicit cold-start path; validate minimum-history rule.
Customer privacy exposure	Raw PII could leak into training or logs.	Pseudonymise analytical records and restrict access; privacy reviewer TBC.
Model optimized only for purchases	More reminders could increase purchases but also annoy customers.	Monitor opt-outs and complaints; keep customer experience as a guardrail.
No named owners or channels	No one can approve purpose, templates or notification API.	Assign product, privacy, OMS/IMS, notification and model owners before pilot.

15. Final go-live checklist

- [ ] Purpose, consent, channel, retention and opt-out policy approved.
- [ ] OMS fulfilled purchases reliably link a permitted customer key to the actual SKU, quantity and timestamp.
- [ ] WMS substitutions/short-picks and returns are accounted for.
- [ ] IMS pack size, unit conversion and availability data are reliable.
- [ ] Depletion event or repeat-purchase proxy is explicitly defined; unobserved events are marked correctly.
- [ ] Customer/product history is sufficient for model training and fair evaluation; otherwise remain in collection/shadow stage.
- [ ] XGBoost Survival AFT is compared with a simple baseline and results are documented.
- [ ] Minimum-history and cold-start behavior are tested.
- [ ] Consent, opt-out, availability, recent order, cart, quiet-hour and frequency checks pass.
- [ ] Duplicate sends, retry behavior, candidate expiry and kill switch pass integration tests.
- [ ] Shadow run is reviewed by product/privacy owners before limited pilot.
- [ ] Monitoring, complaint handling, rollback, support owner and model review schedule are in place.

References and supporting documents

- Accesco, Dual Replenishment Engine Model Architecture (working draft, October 2026) - basis for customer-facing XGBoost Survival AFT, event-time labels and data-readiness gates.
- Accesco, Unified Admin Panel Specification (internal working document) - proposed order and inventory source ownership; confirm any conflicts with the implemented system.
- XGBoost official documentation, Survival Analysis with Accelerated Failure Time: https://xgboost.readthedocs.io/en/latest/tutorials/aft_survival_analysis.html - describes observed and censored event-time records.
- Databricks model lifecycle documentation: https://docs.databricks.com/aws/en/machine-learning/manage-model-lifecycle - confirm capabilities in the contracted workspace.
- Accesco, SRM_IMS_WMS_TMS_Integration.pdf (May 2026) - supporting context for system hand-offs; customer reminder paths still require product and privacy approval.
- All APIs, consent records, notification channels and production source fields are proposed requirements until the relevant Accesco owners confirm them.

Implementation readiness sign-off

Complete this register during kickoff and before the customer pilot. The register does not assign work; it records the approvals that must be obtained.

Decision / approval	Approver role	Status	Date / reference
Reminder purpose, consent and retention	Product owner + privacy reviewer	Yet to be assigned	TBC
Customer history and event definition	OMS owner + analytics lead	Yet to be assigned	TBC
Actual fulfilled quantity and timestamp mapping	OMS/WMS owners	Yet to be assigned	TBC
Product pack-size and unit mapping	IMS owner	Yet to be assigned	TBC
AFT label, censoring and evaluation results	ML/analytics lead	Yet to be assigned	TBC
Notification API, duplicate prevention and opt-out	Backend/notification owner	Yet to be assigned	TBC
Shadow-run review and pilot population	Product owner + privacy reviewer	Yet to be assigned	TBC
Production activation and stop-send procedure	Sponsor + product/privacy owners	Yet to be assigned	TBC